Day 2 – Core Models: Publisher, Game, GameKey

🏷️ Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure the virtual environment is activated (`source .venv/bin/activate`).
   - Have the Django dev server stopped or running in the background.
   - Open `games/models.py` and `games/admin.py` in your IDE.
2. **Prerequisites Checklist**:
   - Students must have completed Day 1 (Django project scaffolded, custom `games` app created, and dev server working).
3. **Required Material**:
   - A diagram or whiteboard showing the database relationships: `User (Django built-in) <1-to-1> Publisher <1-to-many> Game <1-to-many> GameKey <many-to-1 (optional)> User (owner)`

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain Django's ORM mental model (classes/attributes mapped to tables/columns).
- Define relationships using `ForeignKey` and `OneToOneField`.
- Apply cascading rules on relationships using `on_delete=models.CASCADE`.
- Generate and run database migrations using `makemigrations` and `migrate`.
- Register models with Django's administrative interface and manage data through the admin UI.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	20 min	Discuss ORM mental models, design the entities, and draw relationship diagrams.

2. Key Concepts & Core Ideas	15 min	Define fields, relationship types, cascade deletion, and migrations.
3. Live Coding Walkthrough	40 min	Live-code the models, run database migrations, and configure administrative site.
4. Check for Understanding	10 min	Q&A on relationship choices, migrations, and model properties.
5. Class Wrap-up & Checkpoint	5 min	Verify models are editable in the admin UI.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (20 min)

- **Teacher Action**: Draw an entity-relationship diagram on the board. Explain that before writing code or routing APIs, we must build a strong foundation: the database schema.
- **ORM Mental Model**: Explain that Django models are Python classes, class attributes represent database columns, and instances of these classes represent database rows.
- **Relationships**: Explain how our entities connect:
  - **Publisher** has a one-to-one link to Django's built-in **User** (for logging in).
  - **Game** belongs to a single **Publisher** (foreign key).
  - **GameKey** belongs to a single **Game** (foreign key), and optionally has a **User** owner (when purchased).

2. Key Concepts & Core Ideas (15 min)

Introduce these essential terms:

- **ORM (Object-Relational Mapper)**: Bridges the gap between OOP in Python and Relational SQL databases (SQLite for dev, PostgreSQL for prod).
- **`ForeignKey` VS `OneToOneField`**:
  - `ForeignKey` establishes a one-to-many relationship (one publisher can release many games).
  - `OneToOneField` enforces uniqueness (one user can represent only one publisher).
- **`on_delete=models.CASCADE`**: When a parent object is deleted, all child objects referencing it are deleted.
  - **Question**: What should happen to GameKeys if we delete a Game?
    - *Answer*: Since keys depend on a game's existence, deleting the game cascades the deletion, deleting all associated GameKeys. Contrast this with `SET_NULL` (retaining keys with null game fields) or `PROTECT` (blocking game deletion while keys exist).

- **`null=True` VS `blank=True`**: `null` is database-level (allows database cells to be NULL), while `blank` is validation-level (allows empty inputs in forms/serializers).
- **Migrations**: Explain migrations as a version-control system for the database schema.
  - **Question**: Why do we say migrations are versioned schema changes, not magic?
    - *Answer*: Migrations are Python files representing a history of changes. Each script defines the forward schema change and the reverse rollback step, allowing Git to track database schema evolution over time.

---

3. Live Coding Walkthrough (40 min)

Step 3.1: Defining the Models

- **Explain**: Open `games/models.py`. We will define the three core models. Explain why `GameKey.owner` uses `null=True, blank=True` (keys exist before being sold and having an owner).
- **Code**: Modify `games/models.py`:

```
from django.db import models
from django.contrib.auth.models import User


class Publisher(models.Model):
    name = models.CharField(max_length=200)
    webhook_url = models.URLField()
    webhook_secret = models.CharField(max_length=64)
    user = models.OneToOneField(User, on_delete=models.CASCADE)

    def __str__(self):
        return self.name


class Game(models.Model):
    title = models.CharField(max_length=200)
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    price = models.DecimalField(max_digits=6, decimal_places=2)

    def __str__(self):
        return self.title


class GameKey(models.Model):
    STATUS_CHOICES = [('active', 'Active'), ('expired', 'Expired')]

    key_string = models.CharField(max_length=50, unique=True)
    game = models.ForeignKey(Game, on_delete=models.CASCADE)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, default='active')
    expires_at = models.DateTimeField()
    owner = models.ForeignKey(User, on_delete=models.CASCADE, null=True, blank=True)

    def __str__(self):
```

```
        return self.key_string
```

- **Verify**: Run `python manage.py check` to ensure there are no syntax or configuration errors in the models.
- ⚠ **Common Pitfalls**:
  - **Missing `__str__` method**: Show students how the admin UI looks if `__str__` is omitted—it defaults to unhelpful strings like "Publisher object (1)".
  - **Confusing `null` and `blank`**: Students often write `blank=True` but forget `null=True` on nullable database fields, which triggers database integrity errors when saving empty records.

Step 3.2: Database Migrations

- **Explain**: Scan models for modifications and create schema migration instructions, then execute them.
- **Code**:

```
python manage.py makemigrations
python manage.py migrate
```

- **Verify**: Open the generated migration file (e.g., `games/migrations/0001_initial.py`) and read its contents to the class so they understand how Django translates models into schemas.
- ⚠ **Common Pitfalls**:
  - **Forgetting `'games'` app in `INSTALLED_APPS`**: If the app is missing from `settings.py`, `makemigrations` will report "No changes detected".
  - **Running `migrate` before `makemigrations`**: Nothing will happen. Remind students of the two-step migration lifecycle.

Step 3.3: Registering Models in Django Admin

- **Explain**: To inspect and manage our models, we will register them with Django's built-in administration panel.
- **Code**: Modify `games/admin.py`:

```
from django.contrib import admin
from .models import Publisher, Game, GameKey

admin.site.register(Publisher)
admin.site.register(Game)
admin.site.register(GameKey)
```

- **Verify**: Create a superuser and log into the admin panel to test model registration:

```
# Create admin credentials
python manage.py createsuperuser
```

Run the server, navigate to [redacted], log in, and ensure you can create/edit a `Publisher`, a `Game`, and a `GameKey` manually.

admin UI